

```

• <html>
• <head>
• <title>Some HTML in a Python string</title>
• <body>
• "We can use the quotation characters in here without
issues," the author exclaimed.
• Even this one: `
• </body>
• </html>
• """

```

There are also other conveniences afforded by Python for strings. Such as, concatenating two strings using the plus sign, repeating a string multiple times using the multiplication sign (*). Here are a few example:

```

• "You should never " + "ever " * 3 + "divide strings"

```

In Python as with many other languages, strings are arrays of characters, and as such you can use indexing to get to particular characters within the sequence. Python goes a bit further and allows for negative indexing, access sequences of characters within a string and more. Here are a few examples along with their outputs:

```

• a_string = "FastTrack"
• print(a_string[0])      # F
• print(a_string[0:4])    # Fast
• print(a_string[4:])     # Track
• print(a_string[4:])     # Track
• print(a_string[-1])     # k
• print(a_string[::-2])   # FsTak
• print(a_string[::-1])   # kcarTtsaF

```

Here's an overview, to get to a subsequence you can use the `string[start:stop]` syntax; if you omit the start it starts from the beginning, and if you omit the stop it goes till the end. Negative indexing counts characters from the end of the string rather than the beginning. You can provide a stride in the `string[start:stop:stride]` syntax to skip characters; so in `a_string[::-2]` we are selecting every second character starting from the beginning and going to the end. In the final example where we provide a negative stride, it will iterate over the string backwards, thus giving us a reversed string in a single line of code.

Another final bit of string knowledge that can come in handy while building websites is string formatting. Here is how to accomplish that in Python: